Question: 1

When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2022: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

- (●) Require subagents to include publication or data collection dates in their structured outputs. ✓✓
- ( ) Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.
- ( ) Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.
- ( ) Configure the web search agent to only return results from the past 6 months

Explanation:

A. Require subagents to include publication or data collection dates in their structured outputs. Correct.
Providing timestamps allows the synthesis agent to understand that the figures refer to different points in time, enabling it to interpret the data as a trend (growth) rather than a contradiction.
B. Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix. Incorrect.
This approach hides useful context and does not help the agent understand relationships between data points over time.
C. Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric. Incorrect.
Discarding older data removes valuable historical insight and prevents trend analysis.
D. Configure the web search agent to only return results from the past 6 months. Incorrect.
Limiting recency reduces context and does not address the core issue of interpreting time-based differences.

Question: 2

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially complete and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach be Information fidelity with context efficiency when restoring agent state?

- ( ) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. ✓✓
- (●) Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. ×
- ( ) Have each agent maintain its own persistent state file and reload it independently at the start of each session.
- ( ) Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings.

Explanation:

A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. Correct.
This provides **high information fidelity** (structured, complete outputs) while maintaining **context efficiency** (only relevant pieces are re-injected into prompts). The coordinator remains in control of what each agent needs, avoiding unnecessary bloat and duplication.
B. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. ❌
Incorrect.
Conversation logs are often verbose and unstructured, leading to **context overload** and inefficient prompt usage without guaranteed clarity.
C. Have each agent maintain its own persistent state file and reload it independently at the start of each session. ❌ Incorrect.
This decentralizes control and can lead to **inconsistencies and coordination issues**, especially when agents need shared or aligned context.
D. Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings. Incorrect.
Vector stores are useful for retrieval, but they introduce **probabilistic recall** and may miss or distort critical structured state, reducing fidelity during recovery.

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

- ○ Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before Invoking synthesis again. ✓✓
- ◉ Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information. ✗
- ○ Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.
- ○ Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Explanation:
A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again. Correct.
This introduces an **iterative feedback loop**, where identified gaps are actively addressed. The coordinator maintains control and ensures completeness before final report generation.
B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information. Incorrect.
Broader queries may help coverage but are inefficient and still won't guarantee that **specific gaps discovered later** are filled.
C. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output. Incorrect.
This improves transparency but does **not solve the completeness problem**.
D. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator. Incorrect.
This breaks **separation of concerns** and reduces system control. The coordinator should manage task delegation, not the synthesis agent.

Question: 4

Your portfolio value tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning information as a formatted text string. What is the primary advantage of using structured output with defined fields?

- ○ Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.
- ◉ The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations. ✓✓
- ○ Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.
- ○ JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.

Explanation:
A. Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs. ❌ Incorrect.
Token usage depends on the content; JSON is not inherently more compact than text and may sometimes use more tokens.
B. The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations. ✅ Correct.
Structured output provides **clear, predictable fields**, making it easy for the agent to use the data accurately in downstream steps.
C. Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values. ❌ Incorrect.
The model is still probabilistic; JSON improves structure, but not deterministic processing.
D. JSON schemas automatically validate that the underlying API returned correct data before the agent processes it. ❌ Incorrect.
Schemas define structure, but they do not guarantee correctness of the actual data returned by the API.

Question: 3

Your CRM agent's delete_contact tool handles requests like "delete the duplicate entry for Acme Corp." The database contains similarly named records (e.g., "Acme Corp," "Acme Corporation," "ACME Corp Inc."), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

- ○ Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion. ✓✓
- ◉ Require users to supply the exact record ID from the CRM Interface rather than using natural language references to contact names. ×
- ○ Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.
- ○ Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.

Explanation:

A. Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion. Correct.
This directly addresses ambiguity by showing **clear distinctions between similar records** while keeping the workflow fast with a lightweight confirmation step.
B. Require users to supply the exact record ID from the CRM interface rather than using natural language references to contact names. ❌ Incorrect.
This reduces errors but adds significant friction and hurts usability for routine tasks.
C. Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests. ❌ Incorrect.
Helpful as a separate improvement, but it doesn't solve **incorrect deletions during manual requests**.
D. Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow. ❌ Incorrect.
This mitigates impact after errors occur but does not reduce the **error rate itself**.

After Implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats — Inline citations vs. bibliographies, methodology sections vs. details embedded in Introductions. What's the most effective way to address these failures?

- ○ Modify your schema to make citations and methodology optional, and flag Incomplete records for manual review rather than falling validation.
- ◉ Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model falls to extract. ×
- ○ Add few-shot examples demonstrating extractions from documents with varied structures — showing how to identify citations in different formats and locate methodology details across section types. ✓✓
- ○ Implement retry logic that re-sends requests when validation detects empty required fields.

Explanation:

A. Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation. ❌ Incorrect.
This lowers data quality standards and avoids solving the extraction problem.
B. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract. ❌ Incorrect.
Regex approaches are brittle and unreliable across varied formats, especially for complex structures like methodology.
C. Add few-shot examples demonstrating extractions from documents with varied structures — showing how to identify citations in different formats and locate methodology details across section types. ✅ Correct.
This directly improves the model's ability to **generalize across diverse document formats**, addressing the root cause of missed extractions.
D. Implement retry logic that re-sends requests when validation detects empty required fields. ❌ Incorrect.
Retries without improving guidance will likely produce the same incomplete outputs.

Question: 7

Your schema includes a skills: string[] field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract skills mentioned." What's the most effective improvement?

- ○ Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- ◉ Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries. ×
- ○ Enrich the schema to [scondidering] to capture extraction metadata.
- ○ Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity. ✓✓

Explanation:

A. Add constraints: "Extract 10–20 skills maximum, one skill per entry, only explicitly named skills." ❌ Incorrect.
This enforces limits but is **arbitrary** and may exclude valid skills or still leave ambiguity in how to split phrases.
B. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries. ❌ Incorrect.
Helpful downstream, but it does not fix **inconsistent extraction behavior at the source**.
C. Enrich the schema to capture extraction metadata. ❌ Incorrect.
Adds complexity but does not directly address inconsistency in skill identification and formatting.
D. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity. ✅ Correct.
Examples directly guide the model on **how to split, what to include, and the expected level of detail**, addressing all three issues effectively.

Question: 8

Your pipeline uses a tool called extract_metadata with a JSON schema for paper details. You've also defined lookup_citations and verify_doi tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls lookup_citations first, which fails because it needs the DOI that extract_metadata would provide. What's the most effective way to ensure structured metadata extraction happens first?

- ○ Set tool choice to {"type": "tool", "name": "extract_metadata"} and process the enrichment requests in subsequent turns after receiving the extracted metadata. ✓✓
- ◉ Set tool choice to "any" so Claude must use a tool, combined with system prompt instructions prioritizing extract_metadata. ×
- ○ Set tool choice to {"type": "tool", "name": "extract_metadata"} for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- ○ Set tool choice to "auto" and reorder the tool definitions so extract_metadata appears first in the tools array, since Claude prioritizes earlier-listed tools.

Explanation:

A. Set tool choice to {"type": "tool", "name": "extract_metadata"} and process the enrichment requests in subsequent turns after receiving the extracted metadata. ✅ Correct.
This enforces the correct **execution order**, ensuring required data (like DOI) is available before dependent tools are called.
B. Set tool choice to "any" so Claude must use a tool, combined with system prompt instructions prioritizing extract_metadata. ❌ Incorrect.
This does not guarantee ordering — Claude may still choose the wrong tool first.
C. Set tool choice to {"type": "tool", "name": "extract_metadata"} for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur. ❌ Incorrect.
This is too rigid and prevents legitimate use of other tools in later steps.
D. Set tool choice to "auto" and reorder the tool definitions so extract_metadata appears first in the tools array, since Claude prioritizes earlier-listed tools. ❌ Incorrect.
Tool ordering does not reliably control selection or execution order.

Question: 9

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- ○ Submit all documents to the Batch API with custom ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.
- ◉ Submit all documents to the real-time Messages API to ensure consistent processing latency across document types. ×
- ○ Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- ○ Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API. ✓✓

Explanation:

A. Submit all documents to the Batch API with custom ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions. ❌ Incorrect.
Batch processing introduces delays that can exceed the **30-minute requirement**, making it unsuitable for urgent reports.
B. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types. ❌ Incorrect.
This meets latency needs but is unnecessarily expensive for standard reports that don't require real-time processing.
C. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return. ❌ Incorrect.
Hourly batching further increases delay and cannot meet urgent processing requirements.
D. Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API. ✅ Correct.
This balances **cost efficiency and latency requirements**, ensuring urgent reports are processed quickly while optimizing cost for non-urgent ones.

Question: 10

Users report that during extended conversations, the AI loses track of specific topics, examples, and preferences they mentioned earlier in the session. Your current implementation uses a sliding window that keeps only the most recent 25 message pairs to stay within context limits. What's the most effective approach to maintain awareness of earlier conversation content while managing context size?

- ○ Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim. ✓✓
- ○ Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query.
- ◉ Increase the window size to 50 message pairs to retain more conversation history before truncation. ×
- ○ Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation.

Explanation:

A. Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim. ✅ Correct.
This preserves **long-term context in a compressed form** while keeping recent interactions intact, maintaining continuity without exceeding token limits.
B. Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query. ❌ Incorrect.
While powerful, this is more complex and better suited for **large-scale or cross-session retrieval**, not necessarily the most direct fix for in-session context loss.
C. Increase the window size to 50 message pairs to retain more conversation history before truncation. ❌ Incorrect.
This only delays the issue and increases token usage without fundamentally solving it.
D. Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation. ❌ Incorrect.
This can lead to **summary drift and compounding errors**, reducing reliability over time.